

Section 9.1: Platform Lifecycles & Virtual Execution Tiers

Study Guide — 2 files in this series / Section 9.*

Executive Overview

Platform engineering enables developer productivity while maintaining reliability. Understanding IaaS/PaaS/SaaS trade-offs and GitOps patterns is essential for modern cloud architecture. This guide expands each concept with comparative analysis, decision frameworks, and interview-focused design problems.

1. Cloud Abstraction Demarcations: IaaS vs. PaaS vs. SaaS

Conceptual Model

Think of abstraction tiers as a sliding scale of control versus convenience. As you move from IaaS → PaaS → SaaS, you hand more operational responsibility to the provider and gain speed at the cost of flexibility.

Responsibility Transfer Model:



IaaS (Infrastructure-as-a-Service)

Control Level:

Provider manages: Hardware, hypervisor, physical network
You manage: OS, runtime, middleware, apps, patches, scaling

Examples:

AWS EC2 + EBS + VPC
Google Compute Engine + Persistent Disk
Azure Virtual Machines

Use Cases: Lift-and-shift migrations, compliance requiring OS-level control, custom kernel tuning, GPU/specialized hardware workloads.

PaaS (Platform-as-a-Service)

What the provider adds on top of IaaS:

Provider manages: OS updates, runtime versions, scaling policies, infra
You manage: Application code, configuration, data schema

Examples:

Heroku, Cloud Foundry
AWS Elastic Beanstalk
Google App Engine
Azure App Service

Trade-offs:

Benefits:

Faster development
Auto-scaling built in
Built-in monitoring
Lower ops overhead

Limitations:

Vendor lock-in
Limited customization
Runtime version constraints
Less control over infra costs

SaaS (Software-as-a-Service)

Complete managed solution:

Provider manages: Everything – infra, platform, and application
You manage: Configuration, user access, data inputs

Typical architecture:

Multi-tenant databases
API-first design
Configurable via UI/API

Examples:

Slack (messaging), Salesforce (CRM), Gmail (email)

Comparative Analysis: When to Use Each

Factor	IaaS	PaaS	SaaS
Control needed	High	Medium	None
Team ops maturity	High required	Medium	None required
Time-to-deploy	Weeks	Days	Hours
Customization	Full	Limited	Minimal
Cost predictability	Variable	Medium	Subscription
Compliance/audit	Easiest	Moderate	Hardest (shared tenancy)
Vendor lock-in risk	Low	Medium	High

Decision heuristic: - Need to run legacy software with custom OS configs → **IaaS** - Building a new web app, want fast iteration → **PaaS** - Non-core business function (email, CRM, HR) → **SaaS**

Interview Design Problem 1: Choosing an Abstraction Tier

Problem: A fintech startup needs to build a payment processing backend. They have 3 engineers, strict PCI-DSS compliance requirements, and need to launch in 6 weeks. Which tier(s) do you recommend and why?

Solution Framework:

Requirements analysis:

PCI-DSS → need network isolation, audit logs, OS hardening

3 engineers → limited ops capacity

6 weeks → fast deployment needed

Tension:

IaaS gives control for PCI compliance but requires ops expertise

PaaS is faster but may not satisfy all PCI controls

SaaS for non-payment parts (email, analytics) frees capacity

Recommended architecture:

Payment processing core → IaaS (EC2 in dedicated VPC, VPC Flow Logs)

Reason: PCI-DSS requires provable network isolation and OS control

Application servers → PaaS (Elastic Beanstalk)
Reason: Reduce ops burden for non-sensitive app layer
CRM/email → SaaS
Reason: Not in PCI scope, no value in self-managing

Key insight: "Right tier for right workload" – not a single answer.
PCI compliance forces IaaS for the cardholder data environment,
but surrounding services can use higher-abstraction tiers.

2. Workload Virtualization Mechanics

Virtual Machines

Hypervisor Architecture:

Type 1 (Bare metal) – runs directly on hardware:
VMware ESXi, Microsoft Hyper-V, Xen (AWS EC2)
Lower overhead, production-grade

Type 2 (Hosted) – runs on top of OS:
VirtualBox, VMware Workstation
Higher overhead, developer use

Performance characteristics:
Memory overhead: 5-10%
CPU overhead: 2-5%
Boot time: 30-60 seconds
Isolation level: Full kernel separation

Containerization

How Linux namespaces provide isolation:

PID namespace: Process isolation (container can't see host PIDs)
Network namespace: Separate network interfaces, IP, routing tables
Mount namespace: Independent file system view
User namespace: Remapped UIDs (root in container ≠ root on host)

Resource control with cgroups:

CPU: cpu.shares = 512 (default 1024 – gets 50% of a core relative to others)
Memory: memory.limit_in_bytes = 536870912 (512MB hard limit)
I/O: blkio.weight controls disk bandwidth priority

Container runtime stack:

```
Docker:      dockerd → containerd → runc
Kubernetes:  containerd (default) or CRI-O → runc
```

runc is the actual OCI-compliant process that creates the container.
Everything above it is orchestration and image management.

VM vs. Container Comparison

Aspect	VM	Container
Startup	30–60 seconds	< 1 second
Resource overhead	5–10%	1–2%
Isolation strength	Full kernel separation	Namespace-based (shared kernel)
Portability	Lower (image size GBs)	Higher (image size MBs)
Security boundary	Stronger (VM escape is rare)	Weaker (kernel exploits affect all containers)
Use case fit	Untrusted workloads, legacy software	Microservices, CI/CD, cloud-native apps

When containers are NOT the right choice: - Multi-tenant environments with hostile tenants (use VMs or gVisor/Firecracker) - Workloads requiring different kernel versions - GUI applications or legacy software with kernel module dependencies

Interview Design Problem 2: VM vs. Container for a Multi-Tenant SaaS

Problem: You're designing a SaaS platform where customers run arbitrary user-uploaded code (think: online code judges, notebook runners). Should you use VMs or containers? What if you need 1,000 simultaneous executions?

Solution Framework:

```
Risk analysis of user-submitted code:
Arbitrary code execution → adversarial input likely
Container escape via kernel exploit is known attack surface
VMs offer stronger isolation (separate kernel)
```

Constraints:

1,000 simultaneous executions
VM boot time: 30-60 seconds → unacceptable for interactive use
Container startup: <1 second → acceptable

Solution: Micro-VMs (Firecracker)
AWS Firecracker: KVM-based micro-VMs
Boot time: ~125ms (vs 30-60s for full VMs)
Memory overhead: ~5MB per VM
Security: full kernel isolation
Used by: AWS Lambda, Fargate

Architecture:
Pre-warm pool of Firecracker micro-VMs
Assign one per user execution
Destroy immediately after run (no state reuse)
Overlay file system for code injection

Key insight: The "VM vs Container" question has a third option – micro-VMs bridge startup speed and isolation guarantees.
Pure containers are insufficient for hostile multi-tenant execution.

3. Deployment Architecture Strategies

Blue-Green Deployment

Mechanism:

Two identical environments:
Blue → currently serving 100% of production traffic
Green → idle, receives new deployment

Sequence:
1. Deploy v2 to Green
2. Run smoke tests against Green
3. Switch load balancer: 100% traffic → Green
4. Blue becomes standby for instant rollback

Rollback: one load balancer update, instant
Cost: 2× infrastructure (both environments running)

Canary Rollout

Mechanism:

Traffic split evolves gradually:
T+0min: 1% → canary (v2), 99% → stable (v1)
T+5min: 5% → canary

```
T+15min: 25% → canary
T+60min: 100% → canary (v1 decommissioned)
```

Monitoring gates between each step:

```
Error rate delta: < 0.1% vs baseline
p99 latency delta: < 10ms vs baseline
Business metric: No degradation in conversion rate
```

Rolling Updates

Mechanism (Kubernetes default):

Parameters:

```
maxSurge: 25% → can temporarily exceed desired pod count
maxUnavailable: 25% → max pods taken down at a time
```

Sequence (10 pods, maxSurge=2, maxUnavailable=2):

Phase 1: Create 2 new pods (now 12 total)

Phase 2: Terminate 2 old pods (now 10)

Repeat until all pods are new version

Health checks enforced between phases.

No complete downtime, but mixed versions exist simultaneously.

Comparative Analysis: Choosing a Deployment Strategy

Factor	Blue-Green	Canary	Rolling
Rollback speed	Instant (switch LB)	Gradual (flush traffic)	Slow (re-roll)
Infrastructure cost	2x	1.25x	1x
Risk exposure	Binary (all or nothing)	Gradual	All users, incrementally
Version mixing	Never	Brief	Yes, during rollout
Database migration	Requires backward compat	Same	Same
Best for	API/backend changes	Feature experiments, perf	Standard updates

Decision rules: - Strict SLA, financial system, regulatory audit → **Blue-Green** (provable state before/after) - Feature flags, UX changes, performance testing → **Canary** (real user signals) - Standard iterative release with no schema changes → **Rolling**

Interview Design Problem 3: Deployment Strategy for E-Commerce Checkout

Problem: Black Friday is in 2 days. Your team needs to deploy a new checkout flow with a database schema migration (adding a nullable column to `orders`). You serve 100K RPS at peak. Which deployment strategy do you use and how do you handle the migration?

Solution Framework:

Constraints:

- 100K RPS – no tolerance for errors
- Schema migration – new column added (nullable, backward compatible)
- 48 hours to deploy – time pressure

Schema migration strategy (expand-contract pattern):

Step 1 – Expand: Add nullable column, no app change needed

Old code writes: `column = NULL` (default)

New code writes: `column = value`

→ Safe to run before deployment

Step 2 – Deploy: Blue-green to new checkout code

Both old and new code work with expanded schema

Step 3 – Contract (post-deploy): Backfill NULLs, add NOT NULL constraint

Only after 100% traffic on new code

Deployment: Blue-Green for checkout

Reason 1: Cannot have mixed cart states (half old, half new checkout)

Reason 2: Rollback must be instant if payment errors spike

Reason 3: Need to test exact Green environment before switching

Monitoring gate before traffic switch:

Run 5 min load test against Green at 20% synthetic traffic

Assert: zero payment failures, p99 < 200ms

Rollback trigger:

Error rate > 0.5% on checkout → switch LB back to Blue in 30 seconds

Key insight: The schema migration and the deployment are separate events.

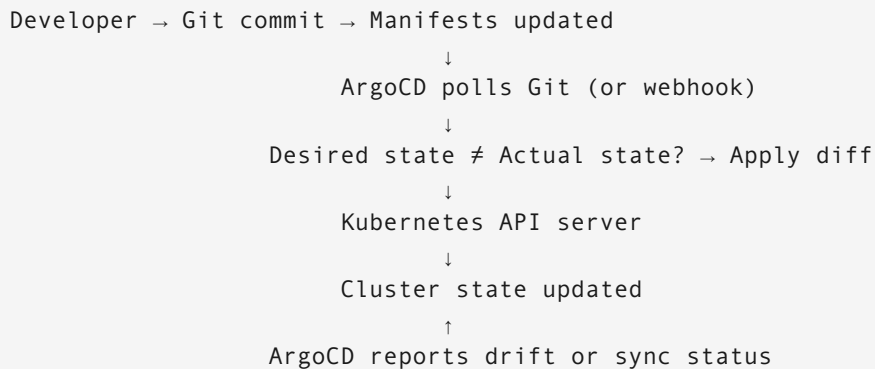
Decouple them: schema first (backward compatible), then blue-green deploy, then schema cleanup. Never migrate schema and code simultaneously.

4. GitOps Architecture

State Convergence Loop

Core principle: Git is the single source of truth. The cluster continuously reconciles toward the desired state expressed in Git.

ArgoCD / Flux pattern:



Drift detection: If someone manually `kubectl apply` s a change, ArgoCD detects the deviation and can auto-revert or alert.

Monorepo Challenges & Solutions

```
monorepo/
├── service-a/
│   ├── deploy/ ← Kubernetes manifests
│   └── src/     ← Application code
├── service-b/
│   ├── deploy/
│   └── src/
└── shared/
    ├── libraries/
    └── common/
```

Problems:

- Long CI build times (all services rebuild on any change)
- Complex ownership boundaries
- Merge conflicts on shared paths
- Large binary artifacts stored in repo

Solutions:

- Build caching (Bazel, Nx, Pants) – only rebuild affected targets
- Selective deployment – path-based triggers in CI
- CODEOWNERS – enforce review requirements per directory
- Separate binary artifact storage (S3, Artifactory, OCI registry)

Interview Design Problem 4 (Bonus — GitOps Focus)

Problem: Your team is adopting ArgoCD. A developer accidentally deletes a Kubernetes deployment directly via `kubectl`. How does GitOps catch and recover from this? What are the risks of auto-sync vs. manual sync?

Solution Framework:

Drift detection:

ArgoCD polls cluster state every 3 minutes (default) or via webhook

Detects: deployment in Git, not in cluster → OutOfSync

Action depends on sync policy:

Auto-sync enabled:

ArgoCD re-applies the manifest automatically

Recovery time: ~3 minutes

Risk: Can fight with a developer doing legitimate manual intervention
(e.g., emergency hotfix) – auto-sync reverts their change

Manual sync:

ArgoCD alerts via Slack/PagerDuty: "App X is OutOfSync"

On-call engineer reviews and approves sync

Recovery time: 5–15 minutes

Safer: human in the loop, but slower

Recommended policy:

Production: Manual sync (human approval gate)

Staging: Auto-sync (fast feedback, acceptable to overwrite)

Additional safeguard: Resource exclusions

Exclude HPA `currentReplicas` from drift detection

Avoid fighting with cluster autoscaler on replica counts

Key insight: Auto-sync is powerful but requires "server-side apply" to handle field management correctly, especially for fields managed by controllers (HPA, cert-manager, etc.).

Summary: Decision Frameworks

Platform Tier Selection

- Use **IaaS** when compliance, custom OS, or legacy software demands full control.
- Use **PaaS** for new applications where speed matters and vendor lock-in is acceptable.
- Use **SaaS** for non-differentiating business functions.

Virtualization Selection

- Use **containers** for cloud-native microservices, CI/CD, and same-tenant environments.
- Use **VMs** for strong isolation, legacy OS requirements, or untrusted workloads.
- Use **micro-VMs** (Firecracker) when you need both startup speed and kernel isolation.

Deployment Strategy Selection

- **Blue-Green:** Backend APIs, payment flows, schema-migrating releases, regulatory deployments.
- **Canary:** Frontend changes, performance experiments, feature rollouts.
- **Rolling:** Routine updates with no version compatibility concerns.
- **Hybrid:** Frontend via canary + backend via blue-green is the production-grade default for complex systems.